

语义下载

[检查资源更新接口](#)

[下载接口](#)

[语义下载过程](#)

[文件下载结果回调](#)

JSON

```
1  语义文件下载策略（pp项目）：
2  1. 下载后文件：customizeOemData.json, customizeSemanticData.json , customizeSlotData.json
3  2. 语义下载路径： "/storage/emulated/10/Android/data/com.zone.service.speech/files/semantic";
4  重定向路径： "/mnt/user/10/emulated/10/Android/data/com.zone.service.speech/files/semantic"
5  3. 文件版本号本地使用setting provider存储，存储类型int型，版本获取方式： settings
   get global customizeOemData|customizeSemanticData|customizeSlotData
6  3. 文件下载过程，
7  3-1： speechservice通过获取本地存储的文件版本号作为接口"/speech/resource/check
   ResourceStatus"入参，返回列表为空表示版本相同无需下载，code码：DownloadErrorCode.
   ERROR_NOT_NEED;
8  3-2： 依据返回结果中的"downloadAddress"字段值拼接url调用下载接口"/speech/file/
   downloadFile"
9  3-3： 下载后文件进行文件大小和md5双重校验，成功后解压文件并删除.gz文件，版本号写入se
   ttings.global
10 4. 异常处理：
11 4-1： 重连机制： 离线状态下请求失败后重连10次，一分钟一次，均失败会设置code码：Downloa
   dErrorCode.ERROR_CANCEL，重置文件verison为0，等待下次连接正常后下载；
12 4-2： 下载后文件大小校验失败 设置code码：DownloadErrorCode.ERROR_FILE_SIZE_NOT
   _MATCH 重置文件verison为0，等待下次连接正常后下载；
13 4-3： 下载后md5校验失败，删除当前单个下载文件，重置文件verison为0，等待下次连接正常
   后重新下载；
14
```

检查资源更新接口

1. 下载路径/storage/emulated/10/Android/data/com.zone.service.speech/files/semantic/
2. baseUrl:<https://speech-pre.z-onesoftware.com/>

```
1 C:\Users\dloucu>adb shell settings get global customizeSemanticData
2 1
3
4 C:\Users\dloucu>adb shell settings get global customizeSlotData
5 1
6
7 C:\Users\dloucu>adb shell settings get global customizeOemData
8 1
9
```

```
1
2 https://speech-pre.z-onesoftware.com/speech/resource/checkResourceStatus
3
4
5 入参body:
6  {
7    "deviceId": "test123",
8    "applicationName": "dataSynchronized",
9    "applicationVersion": "V1.0.0_audi",
10   "resources": [{
11     "resourceName": "customizeOemData_audi_UP.json",
12     "resourceVersion": "V1.0.0_audi",
13     "version": 0,
14     "resourceType": "customize_data"
15   }, {
16     "resourceName": "customizeSemanticData_UP.json",
17     "resourceVersion": "V1.0.0_audi",
18     "version": 0,
19     "resourceType": "customize_data"
20   }, {
21     "resourceName": "customizeSlotData_UP.json",
22     "resourceVersion": "V1.0.0_audi",
23     "version": 0,
24     "resourceType": "customize_data"
25   }]
26 }
27
28 response:
29 {"code":200,"message":"成功","result":[],"updateResource":false}
30 NetworkLog: {"code":30005,"message":"应用版本有误","updateResource":false}
31
32 {
33   "code": 200,
34   "message": "成功",
35   "result": [
36     {
37       "resourceType": "customize_data",
38       "resourceName": "customizeSemanticData_UP.json",
39       "resourceId": "V1.0.0_audi",
40       "resourceMd5": "4cf07d4fa0b47e725fc57f54706a1e3d",
41       "version": 1,
42       "downloadAddress": "dev/customize_data/V1.0.0_audi/customizeSe
43         manticData.gz"
44     },
45     ],
```

```

45     "updateResource": false
46 }
47
48
49 NetworkLog: {"code":200,"message":"成功","result":[{"resourceType":"customize_data","resourceName":"customizeSemanticData_UP.json","resourceId":"V1.0.0_audi","resourceMd5":"20d40c9e7350ed37cb089ae93c0f0ec5","version":1,"downloadAddress":"pre/customize_data/V1.0.0_audi/customizeSemanticData.gz"}, {"resourceType":"customize_data","resourceName":"customizeOemData_audi_UP.json","resourceId":"V1.0.0_audi","resourceMd5":"b0f01dbbed3d870f38e0f09e6fc33db7","version":1,"downloadAddress":"pre/customize_data/V1.0.0_audi/customizeOemData_audi.gz"}, {"resourceType":"customize_data","resourceName":"customizeSlotData_UP.json","resourceId":"V1.0.0_audi","resourceMd5":"cf102ee83d9c9092dea7150d8a1d7a91","version":1,"downloadAddress":"pre/customize_data/V1.0.0_audi/customizeSlotData.gz"}],"updateResource":true}
50 {
51     "code": 200,
52     "message": "成功",
53     "result": [{
54         "resourceType": "customize_data",
55         "resourceName": "customizeSemanticData_UP.json",
56         "resourceId": "V1.0.0_audi",
57         "resourceMd5": "20d40c9e7350ed37cb089ae93c0f0ec5",
58         "version": 1,
59         "downloadAddress": "pre/customize_data/V1.0.0_audi/customizeSemanticData.gz"
60     }, {
61         "resourceType": "customize_data",
62         "resourceName": "customizeOemData_audi_UP.json",
63         "resourceId": "V1.0.0_audi",
64         "resourceMd5": "b0f01dbbed3d870f38e0f09e6fc33db7",
65         "version": 1,
66         "downloadAddress": "pre/customize_data/V1.0.0_audi/customizeOemData_audi.gz"
67     }, {
68         "resourceType": "customize_data",
69         "resourceName": "customizeSlotData_UP.json",
70         "resourceId": "V1.0.0_audi",
71         "resourceMd5": "cf102ee83d9c9092dea7150d8a1d7a91",
72         "version": 1,
73         "downloadAddress": "pre/customize_data/V1.0.0_audi/customizeSlotData.gz"
74     }],
75     "updateResource": true
76 }

```

下载接口

1. <https://speech-pre.z-onesoftware.com/speech/file/downloadFile>

```
JSON |  
1  body:  
2  NetworkLog:  
3  {"deviceId":"test123","path":"pre/customize_data/V1.0.0_audi/customizeSemanticData.gz"}  
4  
5  response:  
6  {  
7    "code": 200,  
8    "message": "成功",  
9    "result": [31, -117, 8, 0],  
10   "size": 141081  
11 }
```

语义下载过程

```

1  try {
2      //      HttpLoggingInterceptor logging = new HttpLoggingInterceptor
        ();
3      //      logging.setLevel(HttpLoggingInterceptor.Level.BODY);
4      //
5      //      Interceptor headerInterceptor = new Interceptor() {
6      //          @Override
7      //          public Response intercept(Chain chain) throws IOException
        {
8          //              Request request = chain.request().newBuilder()
9          //                  .header("Content-Type", "application/json")
10         //                  .build();
11         //              return chain.proceed(request);
12         //          }
13         //      };
14         OkHttpClient client = new OkHttpClient.Builder()
15             .connectTimeout(5000, TimeUnit.MILLISECONDS)
16             .writeTimeout(5000, TimeUnit.MILLISECONDS)
17             .readTimeout(5000, TimeUnit.MILLISECONDS)
18             .addInterceptor(InterceptorUtil.logInterceptor())
19             //          .addInterceptor(logging)
20             //          .addInterceptor(headerInterceptor)
21             .addNetworkInterceptor(InterceptorUtil.headerIntercept
        or())
22             .retryOnConnectionFailure(true)
23             .build();
24
25         Retrofit retrofit = new Retrofit.Builder()
26             .baseUrl(getAppUrl())
27             .addConverterFactory(ScalarsConverterFactory.create())
28             .addConverterFactory(GsonConverterFactory.create())
29             .addCallAdapterFactory(RxJava2CallAdapterFactory.creat
        e())
30             .client(client)
31             .build();
32         mSemanticWebApis = retrofit.create(SemanticWebApis.class);
33     } catch (Exception e) {
34         LogUtils.e(TAG, TAG_PREFIX + "SemanticCloud init error: " + e.
        getMessage());
35     }

```

```
1     private static final int MAX_RETRY_COUNT = 10;
2     private static final long RETRY_DELAY_SECONDS = 60_000L;
3     //     private static final long RETRY_DELAY_SECONDS = 2_000L;
4
5     // 奥迪品牌数据
6     //     private static final String CUSTOMIZE_OEM_SOURCE_NAME = "customizeOemData_audi_UP.json";
7     // 全量品牌数据
8     private static final String CUSTOMIZE_OEM_SOURCE_NAME = "customizeOemData_UP.json";
9     // 全量上线的语义数据
10    private static final String CUSTOMIZE_SEMANTIC_DATA_SOURCE_NAME = "customizeSemanticData_UP.json";
11    // 全量上线的词槽数据
12    private static final String CUSTOMIZE_SLOT_SOURCE_NAME = "customizeSlotData_UP.json";
13    private static final String RESOURCE_VERSION = "V1.0.0_audi";
14    private static final String DATA_SYNCHRONIZED = "dataSynchronized";
15    private static final String CUSTOMIZE_DATA = "customize_data";
16
17    //语义下载路径 "/storage/emulated/10/Android/data/com.zone.service.speech/files/semantic";
18    private static final String SEMANTIC_PATH = "semantic";
19
20    private void preDownloadSemantic() {
21        Map<SpeechGateWayAuthUtil.ServiceName, String> serviceVersion = new HashMap<>();
22        serviceVersion.put(SpeechGateWayAuthUtil.ServiceName.SPEECH, String.valueOf(BuildConfig.VERSION_CODE));
23        serviceVersion.put(SpeechGateWayAuthUtil.ServiceName.XUN_FEI_SDK, String.valueOf(BuildConfig.ENGINE_VERSION));
24        mSemanticCloud.downloadSemantic(serviceVersion, iDownloadResult, semanticVersion, slotVersion, oemVersion);
25    }
26
```

```

1  public void downloadSemantic(Map<SpeechGateWayAuthUtil.ServiceName, String> serviceVersion, IDownloadResult iDownloadResult, int semanticVersion, int slotVersion, int oemVersion) {
2      getDeviceId();
3      initVin();
4      if (TextUtils.isEmpty(mVinCode)) {
5          LogUtils.e(TAG, TAG_PREFIX + "getVinCodeAsync failed! return vinCode is empty");
6          return;
7      }
8      checkResourceStatus(serviceVersion, semanticVersion, slotVersion, oemVersion)
9          .subscribeOn(Schedulers.io())
10         .observeOn(Schedulers.io())
11         .flatMap(checkSemanticResponseBean -> {
12             boolean updateResource = checkSemanticResponseBean.isUpdateResource();
13             List<CheckSemanticResponseBean.ResultDTO> resultDTOList = checkSemanticResponseBean.getResult();
14             if (checkSemanticResponseBean.getCode() == DownloadErrorCode.SUCCESS && updateResource && !resultDTOList.isEmpty()) {
15                 LogUtils.d(TAG, TAG_PREFIX + " batchDownload ");
16                 return batchDownloadSingle(serviceVersion, resultDTOList);
17             } else {
18                 return Single.error(new BusinessException(DownloadErrorCode.ERROR_NOT_NEED, "not need download"));
19             }
20         })
21         .subscribeOn(Schedulers.io())
22         .observeOn(Schedulers.single())
23         .subscribe(fileDownloadResultList -> {
24             LogUtils.d(TAG, TAG_PREFIX + " fileDownloadResultList: checkSemanticResponseBean: " + fileDownloadResultList);
25             boolean updateResult = false;
26             if (null != fileDownloadResultList && !fileDownloadResultList.isEmpty()) {
27                 for (int i = 0; i < fileDownloadResultList.size(); i++) {
28                     DownloadResult downloadResult = fileDownloadResultList.get(i);
29                     updateResult = downloadResult.isUpdateResult();
30                 }
31             }

```



```

32         if (updateResult) {
33             iDownloadResult.onSuccess(fileDownloadResultList);
34         } else {
35             iDownloadResult.onFailed(fileDownloadResultList);
36         }
37     }, iDownloadResult::onFailed);
38 }
39

```

```

1  public Single<CheckSemanticResponseBean> checkResourceStatus(Map<SpeechGateWayAuthUtil.ServiceName, String> serviceVersion, int semanticVersion, int slotVersion, int oemVersion) {
2      Map<String, String> downHeaders = SpeechGateWayAuthUtil.getHeaders(getAppUrl()
3          , P0
4          , ST_CHECK_RESOURCE_STATUS_URL
5          , SpeechGateWayAuthUtil.MethodType.POST
6          , getAppSecretKey(), getAppSecret()
7          , mV
8          , inCode
9          , SpeechGateWayAuthUtil.SpeechModel.NORMAL
10         , s
11         serviceVersion);
12     FsaVersionRequest fsaVersionRequest = getFsaVersionRequest(semanticVersion, slotVersion, oemVersion);
13     String fsaVersionRequestJson = GsonUtils.toJson(fsaVersionRequest);
14     byte[] byteRequest = fsaVersionRequestJson.getBytes(StandardCharsets.UTF_8);
15     RequestBody requestBody = RequestBody.create(MediaType.parse("application/json"), byteRequest);
16     LogUtils.i(TAG, TAG_PREFIX + "fsaVersionRequestJson: " + fsaVersionRequestJson + ", downHeaders: " + downHeaders);
17     return mSemanticWebApis.checkResources(downHeaders, requestBody)
18         .toObservable()
19         .retryWhen(retryObservableFunction())
20         .firstOnError();
21 }

```

```
1 private FsaVersionRequest getFsaVersionRequest(int semanticVersion, int slotVersion, int oemVersion) {
2     List<FsaVersionRequest.Resource> resourceList = new ArrayList<>();
3
4     FsaVersionRequest.Resource resourceOEM = new FsaVersionRequest.Resource();
5     resourceOEM.setResourceName(CUSTOMIZE_OEM_SOURCE_NAME);
6     resourceOEM.setResourceVersion(RESOURCE_VERSION);
7     resourceOEM.setVersion(oemVersion);
8     resourceOEM.setResourceType(CUSTOMIZE_DATA);
9
10    FsaVersionRequest.Resource resourceSemantic = new FsaVersionRequest.Resource();
11    resourceSemantic.setResourceName(CUSTOMIZE_SEMANTIC_DATA_SOURCE_NAME);
12    resourceSemantic.setResourceVersion(RESOURCE_VERSION);
13    resourceSemantic.setVersion(semanticVersion);
14    resourceSemantic.setResourceType(CUSTOMIZE_DATA);
15
16    FsaVersionRequest.Resource resourceSlot = new FsaVersionRequest.Resource();
17    resourceSlot.setResourceName(CUSTOMIZE_SLOT_SOURCE_NAME);
18    resourceSlot.setResourceVersion(RESOURCE_VERSION);
19    resourceSlot.setVersion(slotVersion);
20    resourceSlot.setResourceType(CUSTOMIZE_DATA);
21
22    resourceList.add(resourceOEM);
23    resourceList.add(resourceSemantic);
24    resourceList.add(resourceSlot);
25
26    FsaVersionRequest fsaVersionRequest = new FsaVersionRequest();
27    fsaVersionRequest.setResources(resourceList);
28    fsaVersionRequest.setApplicationName(DATA_SYNCHRONIZED);
29    fsaVersionRequest.setApplicationVersion(RESOURCE_VERSION);
30    fsaVersionRequest.setDeviceId(mDeviceId);
31    fsaVersionRequest.setSpeechApkVersion(SystemPropertyUtil.getSystemVersion());
32    return fsaVersionRequest;
33 }
```

```

1 private Single<List<DownloadResult>> batchDownloadSingle(Map<SpeechGateWayAuthUtil.ServiceName, String> serviceVersion, List<CheckSemanticResponseBean.ResultDTO> resultDTOList) {
2     Single<List<DownloadResult>> listSingle = Observable.fromIterable(
3         resultDTOList)
4         .map(resultDTO -> Pair.create(resultDTO, downloadFileSingle(serviceVersion, resultDTO).blockingGet()))
5         .map(resultDTODownloadResponseBeanPair -> {
6             CheckSemanticResponseBean.ResultDTO resultDTO = resultDTODownloadResponseBeanPair.first;
7             DownloadResult downloadResult = getDownloadResult(resultDTODownloadResponseBeanPair, resultDTO);
8             LogUtils.d(TAG, TAG_PREFIX + ", batchDownload ---downloadResult--- [downloadAddress: " + downloadResult.getDownloadAddress() +
9                 "], [fileName: " + downloadResult.getFileName()
10                + "], [ResourceMd5: " + downloadResult.getResourceMd5() + "], [version: " + downloadResult.getVersion() + "], [size: " +
11                downloadResult.getSize() + "]"");
12             return downloadResult;
13         })
14         .flatMap(this::getFileBytes)
15         .flatMap(this::writeFile)
16         .retryWhen(retryObservableFunction())
17         .toList();
18     return listSingle;
19 }

```

```
1      @NonNull
2      private static DownloadResult getDownloadResult(Pair<CheckSemanticResponseBean.ResultDTO, DownloadResponseBean> resultDTODownloadResponseBeanPair, CheckSemanticResponseBean.ResultDTO resultDTO) {
3          DownloadResponseBean downloadResponseBean = resultDTODownloadResponseBeanPair.second;
4          String downloadAddress = resultDTO.getDownloadAddress();
5          int index = downloadAddress.lastIndexOf('/');
6          String fileName = index == -1 ? "" : downloadAddress.substring(index + 1);
7          DownloadResult downloadResult = new DownloadResult();
8          downloadResult.setDownloadAddress(downloadAddress);
9          downloadResult.setFileName(fileName);
10         downloadResult.setResourceMd5(resultDTO.getResourceMd5());
11         downloadResult.setVersion(resultDTO.getVersion());
12         downloadResult.setSize(downloadResponseBean.getSize());
13         downloadResult.setResult(downloadResponseBean.getResult());
14         return downloadResult;
15     }
```

```
1 public Single<DownloadResponseBean> downloadFileSingle(Map<SpeechGateWayAuthUtil.ServiceName, String> serviceVersion, CheckSemanticResponseBean.ResultDTO resultDTO) {  
2  
3     Map<String, String> downHeaders = SpeechGateWayAuthUtil.getHeaders  
4         (getAppUrl()  
5             ,POST_DOWNLOAD_FILE_URL  
6             ,SpeechGateWayAuthUtil.MethodType.POST  
7             ,getAppSecretKey(), getAppSecret()  
8             ,mVinCode  
9             ,SpeechGateWayAuthUtil.SpeechModel.NORMAL  
10             , serviceVersion);  
11  
12     DownloadRequestBean downloadRequestBean = new DownloadRequestBean(  
13         );  
14     downloadRequestBean.setDeviceId(mDeviceId);  
15     downloadRequestBean.setPath(null != resultDTO.getDownloadAddress()  
16         ? resultDTO.getDownloadAddress() : "");  
17     downloadRequestBean.setSpeechApkVersion(SystemPropertyUtil.getSystemVersion());  
18  
19     String downloadRequestJson = GsonUtils.toJson(downloadRequestBean)  
20     ;  
21     LogUtils.i(TAG, TAG_PREFIX + ", downloadRequestJson:" + downloadRequestJson + ", downHeaders: " + downHeaders);  
22     byte[] byteRequest = downloadRequestJson.getBytes(StandardCharsets.UTF_8);  
23     RequestBody requestBody = RequestBody.create(MediaType.parse("application/json"), byteRequest);  
24  
25     return mSemanticWebApis.downloadFile(downHeaders, requestBody)  
26         .toObservable()  
27         .retryWhen(retryObservableFunction()).firstOnError()  
28         .subscribeOn(Schedulers.io());  
29 }
```

```
1  private ObservableSource<DownloadResult> getFileBytes(DownloadResult down
loadResult) {
2      List<Integer> fileByteResult = downloadResult.getResult();
3      byte[] bytes = FileUtils.convertToByteArray(fileByteResult);
4      downloadResult.setBytes(bytes);
5      return Observable.create(downloadResultObservableEmitter -> {
6          if (downloadResult.getSize() == bytes.length) {
7              downloadResultObservableEmitter.onNext(downloadResult);
8              downloadResultObservableEmitter.onComplete();
9          } else {
10             downloadResultObservableEmitter.onError(new RetryException
(DownloadErrorCode.ERROR_FILE_SIZE_NOT_MATCH, "file size not match", downl
oadResult.getFileName()));
11         }
12     });
13 }
```

```

1  private ObservableSource<DownloadResult> writeFile(DownloadResult downloa
dResult) {
2      List<Integer> fileByteResult = downloadResult.getResult();
3      final boolean[] isValid = {true};
4      if (fileByteResult.isEmpty()) {
5          isValid[0] = false;
6      }
7      byte[] bytes = downloadResult.getBytes();
8      isValid[0] = FileUtils.Bytes2File(bytes, SEMANTIC_PATH, downloa
dResult.getFileName());
9      String md5 = FileUtils.calculateFileMD5(SEMANTIC_PATH, downloa
dResult.getFileName());
10     String resourceMd5 = downloadResult.getResourceMd5();
11     if (md5.equalsIgnoreCase(downloadResult.getResourceMd5())) {
12         FileUtils.unzipGzFile(SEMANTIC_PATH, downloadResult.getFileName
e());
13         isValid[0] = true;
14         downloadResult.setUpdateResult(true);
15     } else {
16         // md5校验失败
17         downloadResult.setUpdateResult(false);
18         FileUtils.deleteFile(SEMANTIC_PATH, downloadResult.getFileName
());
19     }
20     LogUtils.i(TAG, TAG_PREFIX + ", write bytes to file calculateFileM
D5 [Md5InApi: " + resourceMd5 + ", Md5InFile: " + md5 + "], isUpdateResul
t: " + downloadResult.isUpdateResult() + ", fileByteResult size: " + fileB
yteResult.size() );
21
22     return Observable.create(emitter(downloadResult));
23 }
24 private ObservableOnSubscribe<DownloadResult> emitter(DownloadResult d
ownloadResult) {
25     boolean updateResult = downloadResult.isUpdateResult();
26     ObservableOnSubscribe<DownloadResult> observableOnSubscribe = emit
ter -> {
27         if (updateResult) {
28             emitter.onNext(downloadResult);
29             emitter.onComplete();
30         } else {
31             emitter.onError(new RetryException(DownloadErrorCode.ERROR
_MD5_CHECK_FAIL, "file md5 check fail", downloadResult.getFileName()));
32         }
33         LogUtils.d(TAG, TAG_PREFIX + ", write bytes to file: emitter d
ownloadResult: " + downloadResult);

```

```

34         };
35         return observableOnSubscribe;
36     }
37

```

Java

```

1  private static Function<Observable<Throwable>, ObservableSource<?>> retry0
   observableFunction() {
2      final int[] retryCount = {1};
3      return errors -> errors
4          .zipWith(Observable.range(1, MAX_RETRY_COUNT + 1), (throwa
   ble, attempt) -> {
5              if (throwable instanceof IOException || throwable inst
   anceof HttpException || throwable instanceof RetryException) {
6                  if (retryCount[0] < MAX_RETRY_COUNT + 1) {
7                      LogUtils.d(TAG, TAG_PREFIX + ", error, retry af
   ter: " + RETRY_DELAY_SECONDS + "s, retry count: " + retryCount[0]);
8                      retryCount[0]++;
9                      return Observable.timer(RETRY_DELAY_SECONDS, T
   imeUnit.SECONDS);
10                 } else {
11                     BusinessException businessException = new Busi
   nessException(DownloadErrorCode.ERROR_CANCEL, "download cancel");
12                     System.out.println("'" + throwable);
13                     LogUtils.d(TAG, TAG_PREFIX + ", error, retry c
   ancel: " + throwable + ", businessException: " + businessException);
14                     return Observable.error(businessException);
15                 }
16             } else if (throwable instanceof BusinessException) {
17                 LogUtils.d(TAG, TAG_PREFIX + ", error BusinessExce
   ption " + throwable);
18             } else {
19                 LogUtils.d(TAG, TAG_PREFIX + ", other error: " +
   throwable);
20             }
21             return Observable.error(throwable);
22         })
23     .delay(RETRY_DELAY_SECONDS, TimeUnit.MILLISECONDS);
24 }
25
26

```



```
1 private SemanticCloud.IDownloadResult iDownloadResult = new SemanticCloud.
  IDownloadResult() {
2     @Override
3     public void onSuccess(List<DownloadResult> downloadResultList) {
4         //         int downloadVersion = downloadResult.getVersion();
5         //         String fileName = downloadResult.getFileName();
6         LogUtils.d(TAG, TAG_PREFIX + "begin preLoadSemantic..");
7         SemanticFileDataUtils.preLoadSemantic();
8         LogUtils.d(TAG, TAG_PREFIX + "end init SemanticData.");
9         LogUtils.i(TAG, TAG_PREFIX + ", IDownloadResult onSuccess down
loadResultList: " + downloadResultList);
10        //         try {
11        //             if (fileName.contains(CUSTOMIZE_SEMANTIC_DATA_VERSION))
12        //                 Settings.Global.putInt(SpeechServiceApplication.getAppContext().getContentResolver(), CUSTOMIZE_SEMANTIC_DATA_VERSION, downloa
dVersion);
13        //             } else if (fileName.contains(CUSTOMIZE_SLOT_DATA_VERSION)) {
14        //                 Settings.Global.putInt(SpeechServiceApplication.getAppContext().getContentResolver(), CUSTOMIZE_SLOT_DATA_VERSION, downloadVer
sion);
15        //             } else if (fileName.contains(CUSTOMIZE_OEM_DATA_VERSION)) {
16        //                 Settings.Global.putInt(SpeechServiceApplication.getAppContext().getContentResolver(), CUSTOMIZE_OEM_DATA_VERSION, downloadVers
ion);
17        //             }
18        //             LogUtils.d(TAG, "begin preLoadSemantic..");
19        //             SemanticFileDataUtils.preLoadSemantic();
20        //             LogUtils.d(TAG, "end init SemanticData.");
21        //         } catch (Exception e) {
22        //             e.printStackTrace();
23        //         }
24
25    }
26
27    @Override
28    public void onFailed(Throwable throwable) {
29        if (throwable instanceof BusinessException) {
30            int code = ((BusinessException) throwable).getCode();
31            if (code == DownloadErrorCode.ERROR_CANCEL) {
32                LogUtils.d(TAG, TAG_PREFIX + ", IDownloadResult throwa
ble, reset all file" );
33            }
34        }
35    }
36 }
```

```

34         Settings.Global.putInt(SpeechServiceApplication.getApp
Context().getContentResolver(), CUSTOMIZE_SEMANTIC_DATA_VERSION, 0);
35         Settings.Global.putInt(SpeechServiceApplication.getApp
Context().getContentResolver(), CUSTOMIZE_SLOT_DATA_VERSION, 0);
36         Settings.Global.putInt(SpeechServiceApplication.getApp
Context().getContentResolver(), CUSTOMIZE_OEM_DATA_VERSION, 0);
37     } else if (code == DownloadErrorCode.ERROR_FILE_SIZE_NOT_M
ATCH) {
38         String fileName = ((BusinessException) throwable).getF
ileName();
39         LogUtils.d(TAG, TAG_PREFIX + ", IDownloadResult throwa
ble, reset file fileName: " + fileName);
40         //         resetFileVersion(fileName);
41     } else if (code == DownloadErrorCode.ERROR_NOT_NEED) {
42         String fileName = ((BusinessException) throwable).getF
ileName();
43         LogUtils.d(TAG, TAG_PREFIX + ", IDownloadResult throwa
ble, error_not_need fileName: " + fileName);
44     } else {
45         LogUtils.d(TAG, TAG_PREFIX + ", IDownloadResult other
throwable " );
46     }
47     } else if (throwable instanceof UnknownHostException || throwa
ble instanceof NoSuchElementException) {
48         LogUtils.d(TAG, TAG_PREFIX + ", IDownloadResult unknown th
rowable " );
49         Settings.Global.putInt(SpeechServiceApplication.getAppCont
ext().getContentResolver(), CUSTOMIZE_SEMANTIC_DATA_VERSION, 0);
50         Settings.Global.putInt(SpeechServiceApplication.getAppCont
ext().getContentResolver(), CUSTOMIZE_SLOT_DATA_VERSION, 0);
51         Settings.Global.putInt(SpeechServiceApplication.getAppCont
ext().getContentResolver(), CUSTOMIZE_OEM_DATA_VERSION, 0);
52     } else {
53         LogUtils.d(TAG, TAG_PREFIX + ", IDownloadResult onFailed t
hrowable: " + throwable);
54     }
55 }
56
57 @Override
58 public void onFailed(List<DownloadResult> downloadResultList) {
59     LogUtils.e(TAG, TAG_PREFIX + ", iDownloadResult onFailed. dow
nloadResultList: " + downloadResultList);
60     Settings.Global.putInt(SpeechServiceApplication.getAppContext
().getContentResolver(), CUSTOMIZE_SEMANTIC_DATA_VERSION, 0);
61     Settings.Global.putInt(SpeechServiceApplication.getAppContext
().getContentResolver(), CUSTOMIZE_SLOT_DATA_VERSION, 0);
62     Settings.Global.putInt(SpeechServiceApplication.getAppContext
().getContentResolver(), CUSTOMIZE_OEM_DATA_VERSION, 0);

```

63
64

}

};

```
1  public static byte[] convertToByteArray(List<Integer> integerList) {
2      if (integerList == null) {
3          return new byte[0];
4      }
5
6      byte[] byteArray = new byte[integerList.size()];
7      for (int i = 0; i < integerList.size(); i++) {
8          byteArray[i] = integerList.get(i).byteValue();
9      }
10     return byteArray;
11 }
12
13 public static boolean Bytes2File(byte[] bytes, String fileDir, String fileName) {
14     if (null == mContext) {
15         return false;
16     }
17     String absolutePath = mContext.getExternalFilesDir(null).getAbsolutePath();
18     fileDir = absolutePath + File.separator + fileDir;
19     boolean result = true;
20     BufferedOutputStream bos = null;
21     FileOutputStream fos = null;
22     File file = null;
23     if (bytes == null) {
24         return false;
25     }
26     if (bytes.length == 0) {
27         return false;
28     }
29     try {
30         File dir = new File(fileDir);
31         if (!dir.exists()) { // 判断文件目录是否存在
32             dir.mkdirs();
33         }
34         StringBuilder filePath = new StringBuilder(fileDir);
35         filePath.append("/");
36         filePath.append(fileName);
37         file = new File(filePath.toString());
38         if (!file.exists()) {
39             file.createNewFile();
40         }
41         fos = new FileOutputStream(file, false);
42         bos = new BufferedOutputStream(fos);
43         bos.write(bytes);
```

```

44         } catch (Exception e) {
45             e.printStackTrace();
46             result = false;
47         } finally {
48             if (bos != null) {
49                 try {
50                     bos.close();
51                 } catch (IOException e) {
52                     e.printStackTrace();
53                     result = false;
54                 }
55             }
56             if (fos != null) {
57                 try {
58                     fos.close();
59                 } catch (IOException e) {
60                     e.printStackTrace();
61                     result = false;
62                 }
63             }
64         }
65         return result;
66     }
67
68
69     public static String calculateFileMD5(String filePath, String fileName) {
70         if (null == mContext) {
71             return "";
72         }
73         try {
74             String absolutePath = mContext.getExternalFilesDir(null).getAbsolutePath();
75             String fileFullPath = absolutePath + File.separator + filePath + File.separator + fileName;
76
77             LogUtils.i(TAG, TAG_PREFIX + ", calculateFileMD5 fileFullPath: " + fileFullPath);
78             MessageDigest md = MessageDigest.getInstance("MD5");
79             byte[] buffer = new byte[8192];
80             int bytesRead;
81
82             try (InputStream is = new FileInputStream(fileFullPath)) {
83                 while ((bytesRead = is.read(buffer)) != -1) {
84                     md.update(buffer, 0, bytesRead);
85                 }
86             }
87
88             // 将MD5字节数组转换为十六进制字符串

```

```

89         byte[] digest = md.digest();
90         StringBuilder sb = new StringBuilder();
91         for (byte b : digest) {
92             sb.append(String.format("%02x", b));
93         }
94         LogUtils.i(TAG, TAG_PREFIX + ", calculateFileMD5 success: " +
95     sb);
96         return sb.toString();
97     } catch (NoSuchAlgorithmException e) {
98         LogUtils.i(TAG, TAG_PREFIX + ", calculateFileMD5 file not sup
99     port");
100     } catch (IOException e) {
101         LogUtils.i(TAG, TAG_PREFIX + ", calculateFileMD5 file not exi
102     st");
103     } catch (Exception e) {
104         throw new RuntimeException(e);
105     }
106     return "";
107 }
108
109 public static void deleteFile(String semanticPath, String fileName) {
110     if (null == mContext) {
111         LogUtils.d(TAG, TAG_PREFIX + ", file delete fail fileName: "
112     + fileName);
113         return;
114     }
115     String absolutePath = mContext.getExternalFilesDir(null).getAbsol
116     utePath();
117     String fileFullPath = absolutePath + File.separator + semanticPat
118     h + File.separator + fileName;
119     File file = new File(fileFullPath);
120     if (file.exists()) {
121         if (file.delete()) {
122             LogUtils.d(TAG, TAG_PREFIX + ", file delete success: " +
123     fileFullPath);
124         } else {
125             LogUtils.d(TAG, TAG_PREFIX + ", file delete fail: " + fil
126     eFullPath);
127         }
128     } else {
129         LogUtils.d(TAG, TAG_PREFIX + ", file not exist: " + fileFullP
130     ath);
131     }
132 }
133
134 // 解压.gz文件并更改后缀名为.json文件

```

```

128 public static void unzipGzFile(String fileDir, String fileName) {
129     if (null == mContext) {
130         return;
131     }
132     String absolutePath = mContext.getExternalFilesDir(null).getAbsolute
133     Path();
134     String gzFilePath = absolutePath + File.separator + fileDir + File
135     .separator + fileName;
136     // String gzFilePath = fileDir + File.separator + fileName;
137     LogUtils.d(TAG, TAG_PREFIX + ", unzipGzFile gzFilePath: " + gzFil
138     ePath);
139     try {
140         File gzFile = new File(gzFilePath);
141         String tempPath = gzFilePath.substring(0, gzFilePath.lastInde
142         xOf("."));
143         File tempFile = new File(tempPath);
144         GZIPInputStream gzin = new GZIPInputStream(new FileInputStream
145         m(gzFile));
146         FileOutputStream fos = new FileOutputStream(tempFile);
147         byte[] buffer = new byte[1024 * 8];
148         int length;
149         while ((length = gzin.read(buffer)) != -1) {
150             fos.write(buffer, 0, length);
151         }
152         fos.flush();
153
154         String jsonPath = tempPath + ".json";
155         File jsonFile = new File(jsonPath);
156         Files.move(tempFile.toPath(), jsonFile.toPath(), StandardCopyO
157         ption.REPLACE_EXISTING, StandardCopyOption.ATOMIC_MOVE);
158         deleteFile(gzFilePath);
159     } catch (Exception e) {
160         e.printStackTrace();
161         deleteFile(gzFilePath);
162     }
163 }

```

文件下载结果回调


```

1 private SemanticCloud.IDownloadResult iDownloadResult = new SemanticCloud.
  IDownloadResult() {
2     @Override
3     public void onSuccess(List<DownloadResult> downloadResultList) {
4         //         int downloadVersion = downloadResult.getVersion();
5         //         String fileName = downloadResult.getFileName();
6         LogUtils.d(TAG, TAG_PREFIX + "begin preLoadSemantic..");
7         SemanticFileDataUtils.preLoadSemantic();
8         LogUtils.d(TAG, TAG_PREFIX + "end init SemanticData.");
9         LogUtils.i(TAG, TAG_PREFIX + ", IDownloadResult onSuccess down
loadResultList: " + downloadResultList);
10        //         try {
11        //             if (fileName.contains(CUSTOMIZE_SEMANTIC_DATA_VERSION))
12        //                 Settings.Global.putInt(SpeechServiceApplication.getAppContext().getContentResolver(), CUSTOMIZE_SEMANTIC_DATA_VERSION, downloa
dVersion);
13        //             } else if (fileName.contains(CUSTOMIZE_SLOT_DATA_VERSION)) {
14        //                 Settings.Global.putInt(SpeechServiceApplication.getAppContext().getContentResolver(), CUSTOMIZE_SLOT_DATA_VERSION, downloadVer
sion);
15        //             } else if (fileName.contains(CUSTOMIZE_OEM_DATA_VERSION)) {
16        //                 Settings.Global.putInt(SpeechServiceApplication.getAppContext().getContentResolver(), CUSTOMIZE_OEM_DATA_VERSION, downloadVers
ion);
17        //             }
18        //             LogUtils.d(TAG, "begin preLoadSemantic..");
19        //             SemanticFileDataUtils.preLoadSemantic();
20        //             LogUtils.d(TAG, "end init SemanticData.");
21        //         } catch (Exception e) {
22        //             e.printStackTrace();
23        //         }
24
25    }
26
27    @Override
28    public void onFailed(Throwable throwable) {
29        if (throwable instanceof BusinessException) {
30            int code = ((BusinessException) throwable).getCode();
31            if (code == DownloadErrorCode.ERROR_CANCEL) {
32                LogUtils.d(TAG, TAG_PREFIX + ", IDownloadResult throwa
ble, reset all file" );
33

```

```

34         Settings.Global.putInt(SpeechServiceApplication.getApp
Context().getContentResolver(), CUSTOMIZE_SEMANTIC_DATA_VERSION, 0);
35         Settings.Global.putInt(SpeechServiceApplication.getApp
Context().getContentResolver(), CUSTOMIZE_SLOT_DATA_VERSION, 0);
36         Settings.Global.putInt(SpeechServiceApplication.getApp
Context().getContentResolver(), CUSTOMIZE_OEM_DATA_VERSION, 0);
37     } else if (code == DownloadErrorCode.ERROR_FILE_SIZE_NOT_M
ATCH) {
38         String fileName = ((BusinessException) throwable).getF
ileName();
39         LogUtils.d(TAG, TAG_PREFIX + ", IDownloadResult throwa
ble, reset file fileName: " + fileName);
40         // resetFileVersion(fileName);
41     } else if (code == DownloadErrorCode.ERROR_NOT_NEED) {
42         String fileName = ((BusinessException) throwable).getF
ileName();
43         LogUtils.d(TAG, TAG_PREFIX + ", IDownloadResult throwa
ble, error_not_need fileName: " + fileName);
44     } else {
45         LogUtils.d(TAG, TAG_PREFIX + ", IDownloadResult other
throwable " );
46     }
47     } else if (throwable instanceof UnknownHostException || throwa
ble instanceof NoSuchElementException) {
48         LogUtils.d(TAG, TAG_PREFIX + ", IDownloadResult unknown th
rowable " );
49         Settings.Global.putInt(SpeechServiceApplication.getAppCont
ext().getContentResolver(), CUSTOMIZE_SEMANTIC_DATA_VERSION, 0);
50         Settings.Global.putInt(SpeechServiceApplication.getAppCont
ext().getContentResolver(), CUSTOMIZE_SLOT_DATA_VERSION, 0);
51         Settings.Global.putInt(SpeechServiceApplication.getAppCont
ext().getContentResolver(), CUSTOMIZE_OEM_DATA_VERSION, 0);
52     } else {
53         LogUtils.d(TAG, TAG_PREFIX + ", IDownloadResult onFailed t
hrowable: " + throwable);
54     }
55 }
56
57 @Override
58 public void onFailed(List<DownloadResult> downloadResultList) {
59     LogUtils.e(TAG, TAG_PREFIX + ", iDownloadResult onFailed. dow
nloadResultList: " + downloadResultList);
60     Settings.Global.putInt(SpeechServiceApplication.getAppContext(
).getContentResolver(), CUSTOMIZE_SEMANTIC_DATA_VERSION, 0);
61     Settings.Global.putInt(SpeechServiceApplication.getAppContext(
).getContentResolver(), CUSTOMIZE_SLOT_DATA_VERSION, 0);
62     Settings.Global.putInt(SpeechServiceApplication.getAppContext(
).getContentResolver(), CUSTOMIZE_OEM_DATA_VERSION, 0);

```

63
64

}

};